Implementação e Avaliação Experimental das Organizações MIPS32 Monociclo (O1) e Pipeline de 5 Estágios (O2)

Alonso Fernandes Cavalcante[1], **Gwylherme Mota Rodrigues**[1]

[1]Departamento de Ciência da Computação — Universidade Federal de Roraima (DCC/UFRR)
Boa Vista — RR — Brasil

Resumo. Este trabalho apresenta o projeto em nível de transferência entre registradores (RTL) em VHDL-2008, a verificação funcional e a avaliação experimental de desempenho de duas organizações microarquiteturais para um subconjunto de 10 instruções do MIPS32: uma organização de linha de base Monociclo (***O1***) e uma organização em Pipeline escalar de 5 estágios (***O2***) sem encaminhamento (*forwarding*), equipada com Unidade de Detecção de Conflitos (dados RAW, estruturais e de controle) e semântica de aborto e congelamento arquitetural. A corretude funcional é validada contra o simulador MARS 4.5 por meio de um testbench único (`tb_cpu.vhd`). Com base na regra única de caminho crítico e nos atrasos de componentes de **Patterson e Hennessy (2017)** (Seção 4.5, Figura 4.27), obtêm-se períodos de relógio de 0,80 ns para ***O1*** e 0,20 ns para ***O2*** (razão assintótica de clock de 4,00×). Os experimentos sobre o traço principal `T0` (31 instruções úteis) e a Série `S` (`S5` a `S200`) demonstram um *speedup* em tempo de execução que evolui de 2,40× (`S5`) a 3,92× (`S200`), quantificando o impacto do desequilíbrio entre estágios e da sobrecarga fixa de 4 ciclos de enchimento e esvaziamento do pipeline.

1. Introdução

O paralelismo em nível de instrução explorado pela técnica de *pipelining* constitui um dos fundamentos do projeto de processadores modernos (**Patterson e Hennessy, 2017**). Enquanto uma organização monociclo executa integralmente cada instrução em um único ciclo de relógio — cujo período mínimo é penalizado pela soma em série dos atrasos de todas as unidades funcionais ao longo do caminho crítico —, uma organização em pipeline subdivide o caminho de dados em estágios sequenciais isolados por registradores de estágio. Essa subdivisão permite sobrepor temporalmente a execução de múltiplas instruções e reduzir o período de relógio ao atraso do estágio mais lento.

Este relatório documenta as decisões de projeto microarquitetural, a metodologia de avaliação de desempenho e a análise crítica dos resultados obtidos na implementação das organizações **O1 (Monociclo)** e **O2 (Pipeline de 5 estágios: IF, ID, EX, MEM, WB)** em VHDL-2008, atendendo integralmente aos requisitos R1 a R7 da especificação da disciplina de Arquitetura e Organização de Computadores.

2. Arquitetura das Organizações O1 e O2

2.1. Subconjunto da ISA e Componentes Compartilhados (Requisitos R1 e R2)

Ambas as organizações implementam exatamente o mesmo subconjunto de 10 instruções do MIPS32: (i) formato R (`opcode = 000000`): `add` (`funct = 0x20`), `sub` (`0x22`), `and` (`0x24`), `or` (`0x25`) e `slt` (`0x2A`); (ii) formato I aritmético: `addi` (`opcode = 001000`); (iii) acesso à memória: `lw` (`opcode = 100011`) e `sw` (`opcode = 101011`); (iv) desvio condicional relativo a `PC + 4`: `beq` (`opcode = 000100`); e (v) parada: `break` (`opcode = 000000`, `funct = 0x0D`), que encerra a execução sem modificar registradores ou memória e estaciona o `PC` no próprio endereço da instrução `break`.

Para assegurar comparabilidade estrita, **O1** e **O2** compartilham os mesmos módulos combinacionais e de armazenamento no diretório `comum/`: ULA de 32 bits (`ula.vhd`), decodificador secundário da ULA (`alu_control.vhd`), extensor de sinal de 16 para 32 bits (`sign_ext.vhd`), banco de 32 registradores de 32 bits com `$zero` fixo em zero (`banco_reg.vhd`), memória de instruções (`instruction_memory.vhd`) e memória de dados (`data_memory.vhd`). Ambas as memórias possuem capacidade de **1024 palavras de 32 bits (4 KB)**, endereçadas por `addr(11 downto 2)` com dobramento modular (*wrap-around*) para endereços superiores a 4 KB.

2.2. Escolha da Linha de Base O1: Monociclo vs. Multiciclo

Adotou-se a organização **Monociclo** como linha de base (**O1**) em vez de uma organização Multiciclo. Uma implementação Multiciclo exigiria uma máquina de estados finitos (FSM) de controle e apresentaria um CPI variável conforme a instrução (3 ciclos para `beq`, 4 ciclos para tipo-R/`addi`/`sw` e 5 ciclos para `lw`), mesclando variações de CPI com variações de frequência de clock. O Monociclo mantém CPI útil unitário constante (totalizando $N + 1$ ciclos para N instruções úteis mais a instrução `break`), servindo exclusivamente como linha de base de ciclos e tempo de execução para cálculo de *speedup*, enquanto o simulador **MARS 4.5** atua como oráculo funcional externo.

2.3. Pipeline O2, Registradores Entre Estágios e Escrita em Meio Ciclo

A organização **O2** (`o2/cpu_top.vhd`) particiona o caminho de dados nos cinco estágios clássicos (**IF, ID, EX, MEM, WB**), separados por quatro registradores síncronos que totalizam **426 bits**, conforme detalhado na Tabela 1.

Tabela 1: Especificação e largura em bits dos registradores entre estágios em O2.

Registrador	Rastreio (`valid`, `pc`, `break`)	Controle da ISA	Barramentos de Dados e Endereços	Total
IF/ID	33 bits (`valid`, `pc`)	0 bits	`id_instr` (32 bits)	**65 bits**
ID/EX	34 bits (`valid`, `pc`, `break`)	8 bits	`rd1` (32b), `rd2` (32b), `imm32` (32b), `wa` (5b), `funct` (6b)	**149 bits**
EX/MEM	34 bits (`valid`, `pc`, `break`)	4 bits	`alu_result` (32b), `rd2` (32b), `wa` (5b)	**107 bits**
MEM/WB	34 bits (`valid`, `pc`, `break`)	2 bits	`alu_result` (32b), `mem_data` (32b), `wa` (5b)	**105 bits**

Registrador	Rastreio (`valid`, `pc`, `break`)	Controle da ISA	Barramentos de Dados e Endereços	Total
Soma Global	**135 bits**	**14 bits**	**277 bits**	**426 bits**

Seguindo a organização clássica de **Patterson e Hennessy (2017)**, o módulo `banco_reg.vhd` realiza a escrita do estágio `WB` na **borda de descida do clock (`falling_edge(clk)`, 1ª metade do ciclo)** e a leitura combinacional do estágio `ID` na **2ª metade do ciclo**. Graças a essa escrita em meio ciclo, uma instrução produtora no estágio `WB` (distância 3 em relação à instrução consumidora em `ID`) atualiza o registrador na metade do ciclo e a consumidora em `ID` lê o valor já atualizado antes da borda de subida seguinte. Consequentemente, apenas dependências de distância 1 (`EX`) e distância 2 (`MEM`) configuram conflito de dados RAW.

2.4. Unidade de Detecção de Hazards e Congelamento (Requisito R5)

Como **O2** opera sem *forwarding*, *stall*, *flush* ou predição de desvio, a Unidade de Detecção de Hazards monitora continuamente três condições:

- **Conflito de Dados — RAW (`hazard_type = "01"`):** Disparado quando a instrução válida em `ID` lê `rs` (`id_uses_rs = '1'`) ou `rt` (`id_uses_rt = '1'`) e o registrador lido coincide com o registrador destino `wa ≠ $zero` de uma instrução anterior válida com `reg_write = '1'` nos estágios **EX (distância 1)** ou **MEM (distância 2)**. Registra `hazard_pc = id_pc` (endereço da instrução consumidora).
- **Conflito Estrutural (`hazard_type = "10"`):** Avaliado quando o genérico `MEM_UNIFICADA = true`. Ocorre quando uma instrução válida em `MEM` acessa a memória (`mem_read = '1'` ou `mem_write = '1'`) no mesmo ciclo em que `IF` busca uma instrução válida (`if_valid = '1'`). Registra `hazard_pc = pc_reg` (instrução mais nova em `IF`).
- **Conflito de Controle (`hazard_type = "11"`):** O desvio condicional `beq` é resolvido no estágio **EX** (`ex_take_branch = ex_valid and ex_branch and ex_alu_zero`). Quando o desvio é **tomado** e já existem instruções buscadas sequencialmente em `ID` ou `IF`, aciona-se o aborto com `hazard_pc = ex_pc`. Desvios **não tomados** (`ex_alu_zero = '0'`) não provocam conflito, pois as instruções já buscadas em `PC + 4` e `PC + 8` são exatamente as instruções corretas do caminho sequencial.

Ao detectar qualquer conflito, `hazard_abort` vai a `'1'`, os sinais `hazard_type`, `hazard_pc` e `hazard_cycle` são travados e `freeze_pipe = '1'` inibe simultaneamente a escrita no Banco de Registradores, a escrita na Memória de Dados, o avanço do `PC` e a atualização dos quatro registradores entre estágios.

3. Metodologia Experimental e Ameaças à Validade

3.1. Regra Única do Período de Clock e Verificação de Meio Ciclo

Para atender ao Requisito R6 com o mesmo critério em ambas as organizações, aplica-se a seguinte regra única: *o período mínimo de clock (T_{clk}) de qualquer organização equivale ao atraso do caminho combinacional mais longo entre dois elementos de estado consecutivos somado ao overhead de registrador ($T_{caminho_max} + t_{reg}$), respeitando a restrição de meio ciclo do banco de registradores ($T_{clk} / 2 \geq max(t_{RegWrite}, t_{RegRead})$).*

Extraem-se os atrasos dos estágios diretamente da **Figura 4.27 (Seção 4.5, p. 273)** de **Patterson e Hennessy (2017)**, sob a hipótese explícita do livro-texto de t_{reg} = 0 ps: Busca na Memória de Instruções (`IF`) = 200 ps; Leitura do Banco de Registradores (`ID`) = 100 ps; Operação da ULA (`EX`) = 200 ps; Acesso à Memória de Dados (`MEM`) = 200 ps; e Escrita no Banco de Registradores (`WB`) = 100 ps. Aplicando a regra única:

O1 (Monociclo — caminho crítico `lw`): $T_{clk,\,O1} = 200 + 100 + 200 + 200 + 100 =$ **800 ps (0,80 ns)** $\Rightarrow f_{max,\,O1} = 1{,}25$ GHz

O2 (Pipeline 5 estágios — estágio mais lento): $T_{clk,\,O2} = \max(200, 100, 200, 200, 100) =$ **200 ps (0,20 ns)** $\Rightarrow f_{max,\,O2} = 5{,}00$ GHz

Verificação da Restrição de Meio Ciclo em O2: Com $T_{clk,\,O2} = 200$ ps, metade do período vale $T_{clk,\,O2} / 2 =$ **100 ps**. Na 1ª metade do ciclo (0 a 100 ps, concluindo na borda de descida), a escrita do estágio `WB` exige $t_{RegWrite} = 100$ ps ≤ 100 ps. Na 2ª metade do ciclo (100 a 200 ps), a leitura combinacional do estágio `ID` exige $t_{RegRead} = 100$ ps ≤ 100 ps. Ambas as operações cabem exatamente em meio período.

3.2. Ameaças à Validade

1. **Validade Interna — Compartilhamento do Banco de Registradores de Meio Ciclo com O1:** Para reutilizar a mesma entidade `comum/banco_reg.vhd` e manter um ponto único de amostragem no testbench (`falling_edge(clk)`), o RTL de **O1** também realiza a escrita na borda de descida. Na simulação comportamental RTL, os sinais propagam em *delta cycles*; porém, em síntese física real, exigir que todo o caminho crítico de `lw` (800 ps) estabilizasse até a metade do ciclo forçaria $T_{clk,\,O1} / 2 \geq 800$ ps ($T_{clk,\,O1} = 1{,}60$ ns). Adotou-se na análise o valor clássico de 0,80 ns (captura no fim do ciclo em **O1**) para não inflar artificialmente o *speedup* de **O2**.
2. **Validade Interna — Hipótese Ideal $t_{reg} = 0$ ps:** O modelo analítico da Figura 4.27 despreza tempos de *setup*, propagação *clk-to-q* dos registradores de estágio e *clock skew*, que na implementação física reduziriam marginalmente a frequência máxima de **O2**.
3. **Validade Externa — Escalonamento Estático da Carga de Trabalho:** Para evitar aborto em **O2**, tanto o traço `T0` quanto a Série `S` (gerada com blocos variados contendo todas as classes da ISA: `addi`, `sw`, `lw`, `beq` não tomado e instruções tipo-R `add`, `sub`, `or`, `slt`, `and`) respeitam distância mínima de 3 instruções entre escrita e leitura.

4. Resultados Experimentais e Análise Crítica

4.1. Verificação dos Conflitos (TD, TE, TC) e Desempenho na Série S

As Tabelas 2 e 3 apresentam os resultados obtidos na execução automatizada via `./run_all.sh` (GHDL com padrão IEEE 1076-2008), onde todos os 18 cenários encerraram com **0 erros** na verificação automática dos 32 registradores contra os valores do oráculo MARS 4.5.

Tabela 2: Registro de verificação dos traços de conflito na organização O2.

Traço	MEM_UNIFICADA	Conflito Esperado	hazard_type	hazard_pc Obtido	hazard_cycle	Erros
tracos/td.hex	false	Dados — RAW (distância 1)	01 (1)	0x00000004 (ID)	Ciclo 3	0
tracos/te.hex	true	Estrutural (IF + MEM simultâneos)	10 (2)	0x0000000C (IF)	Ciclo 4	0
tracos/te.hex	false	Nenhum (Harvard separada)	00 (0)	— (Concluído)	9 ciclos	0
tracos/tc.hex	false	Controle (beq tomado em EX)	11 (3)	0x00000000 (EX)	Ciclo 3	0

Tabela 3: Métricas consolidadas para T0 e Série S ($T_{clk,O1}$ = 0,80 ns; $T_{clk,O2}$ = 0,20 ns; Razão de Clock = 4,00×; Ideal k = 5,00×).

Traço	$N_{úteis}$	Ciclos O1	Ciclos O2	CPI O1	CPI O2	IPC O2	Tempo O1 (ns)	Tempo O2 (ns)	Ocup. O2 (IF..WB)	Speedup Medido
S5	5	6	10	1,2000	2,0000	0,5000	4,800	2,000	60,00%	**2,4000×**
S10	10	11	15	1,1000	1,5000	0,6667	8,800	3,000	73,33%	**2,9333×**
S20	20	21	25	1,0500	1,2500	0,8000	16,800	5,000	84,00%	**3,3600×**
T0	31	32	36	1,0323	1,1613	0,8611	25,600	7,200	88,89%	**3,5556×**
S50	50	51	55	1,0200	1,1000	0,9091	40,800	11,000	92,73%	**3,7091×**
S100	100	101	105	1,0100	1,0500	0,9524	80,800	21,000	96,19%	**3,8476×**
S200	200	201	205	1,0050	1,0250	0,9756	160,800	41,000	98,05%	**3,9220×**

4.2. Análise Crítica: Speedup Ideal (k = 5) vs. Razão de Clock (4,0×) vs. Speedup Medido

A análise conjunta dos dados da Tabela 3 esclarece três comportamentos fundamentais da microarquitetura:

1. **Desequilíbrio dos Estágios (Speedup Ideal k = 5,00× vs. Razão de Clock = 4,00×):** O ganho teórico clássico igual ao número de estágios (k = 5) só é atingível se todos os 5 estágios dividirem igualmente o caminho crítico monociclo (800 ps / 5 = 160 ps por estágio). Na organização **O2**, os estágios não são equilibrados: `IF`, `EX` e `MEM` requerem 200 ps, enquanto `ID` e `WB` requerem 100 ps. Como o relógio do pipeline é nivelado pelo estágio mais lento (200 ps), há uma folga ociosa de 100 ps em `ID` e 100 ps em `WB` a cada ciclo, reduzindo o limite assintótico de ganho de frequência de 5,00× para a razão de clock $T_{clk,\ O1}$ / $T_{clk,\ O2}$ = 800 / 200 = **4,00×**.

2. **Efeito do Enchimento e Esvaziamento (Speedup Medido vs. Tamanho do Traço N):** Para um traço limpo de N instruções úteis (+ 1 `break`), **O1** demanda exatamente $C_{O1} = N + 1$ ciclos, ao passo que **O2** demanda $C_{O2} = N + 5$ ciclos (4 ciclos adicionais para que o `break` atravesse o pipeline até o estágio `WB`). Como cada uma das $N + 1$ instruções permanece 1 ciclo em cada um dos 5 estágios, a ocupação individual de cada estágio (`IF`, `ID`, `EX`, `MEM`, `WB`) e da via é idêntica e igual a $(N + 1) / (N + 5)$. Assim, o speedup medido segue exatamente a lei analítica:

$$\text{Speedup}_{\text{medido}}(N) = (T_{\text{clk, O1}} / T_{\text{clk, O2}}) \times ((N + 1) / (N + 5)) = 4{,}00 \times ((N + 1) / (N + 5))$$

Para `S5` ($N = 5$), os 4 ciclos de esvaziamento correspondem a 40% do total de ciclos, limitando a ocupação a 60,00% e o speedup a **2,4000×**. À medida que N cresce para `S200` ($N = 200$), a sobrecarga fixa de 4 ciclos dilui-se (< 2%), elevando a ocupação para **98,05%**, reduzindo o CPI de **O2** para **1,0250** e levando o speedup medido a **3,9220×** (98,05% da assíntota de clock de 4,00×).

5. Conclusão

As organizações **O1 (Monociclo)** e **O2 (Pipeline de 5 estágios)** foram integralmente projetadas em VHDL-2008 e validadas sem erros em 18 cenários automatizados (`./run_all.sh`). A Unidade de Detecção de Hazards identificou com exatidão de ciclo e endereço os conflitos RAW, estrutural e de controle, enquanto a avaliação na Série `S` comprovou quantitativamente o impacto do desequilíbrio entre estágios e da latência de enchimento/esvaziamento sobre o desempenho real do pipeline.

Referências

PATTERSON, D. A.; HENNESSY, J. L. (2017). *Computer Organization and Design: The Hardware/Software Interface* (Organização e Projeto de Computadores). Morgan Kaufmann / Elsevier, 5ª edição. Seção 4.5, Figura 4.27, p. 273.